

MASTER

THE ART OF DEVOPS AUTOMATION

TERRAFORM

INFRASTRUCTURE AS CODE



Terraform lets you define, provision and manage cloud infrastructure in a safe, repeatable and efficient way.

Write Once,
Deploy Anywhere!



- ✓ Version Controlled
- ✓ Reproducible
- ✓ Collaborative
- ✓ Scalable
- ✓ Automated

- In this series, you will learn -

- ? What is Terraform?
- ↓ Installation Guide
- ↻ Terraform Workflow
- 📦 Providers & Resources
- 📄 Terraform Configuration
- (x) Variables & Outputs
- 🗄️ State Management
- 🧩 Modules

- 🔍 Data Sources
- ⚙️ Provisioners
- 📄 Terraform CLI Commands
- 📁 Workspaces
- 🗑️ Remote State (S3 + DynamoDB)
- 🛡️ Best Practices
- 🚀 Real World Project

& much more...



Automate Today,
Orchestrate Tomorrow.



WHAT IS TERRAFORM?

2



Terraform lets you describe your infrastructure in code, so you can provision and manage it predictably.



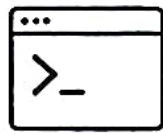
Terraform is an Infrastructure as Code (IaC) tool that helps you build, change and version cloud and on-prem resources safely and efficiently.

HOW DOES TERRAFORM WORK?



1. WRITE

You define your infrastructure in .tf files (HCL - HashiCorp Configuration Language)



2. PLAN

Terraform creates an execution plan to show what will be created, changed or destroyed.



3. APPLY

You approve the plan and Terraform provisions or updates the infrastructure.



4. MANAGE

Terraform tracks the real state of your infra and keeps it in sync with your code.

CORE BENEFITS



Consistency
Same environment every time, everywhere.



Automation
Reduce manual work and human errors.



Scalability
Easily scale your infrastructure up or down.



Speed
Provision resources in minutes, not hours or days.



Version Control
Track changes, roll back and collaborate effectively.

KEY TERMINOLOGY



Configuration
- .tf files where you define the desired infrastructure.



Resource
A single infrastructure object like an EC2 instance, S3 bucket, VPC, etc.



Provider
A plugin that lets Terraform interact with APIs of platforms like AWS, Azure, GCP, etc.



State
A file that stores the current state of your infrastructure managed by Terraform.



Plan
An execution plan that shows what Terraform will do.



Apply
Executes the plan and creates or updates the infrastructure.



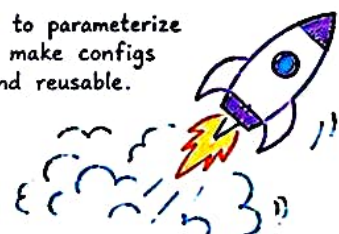
Module
A collection of resources used together. Helps in reusability and abstraction.



Variable
Allows you to parameterize values and make configs dynamic and reusable.



Terraform turns your infrastructure into code, and your code into reliable infrastructure.



WHY TERRAFORM?

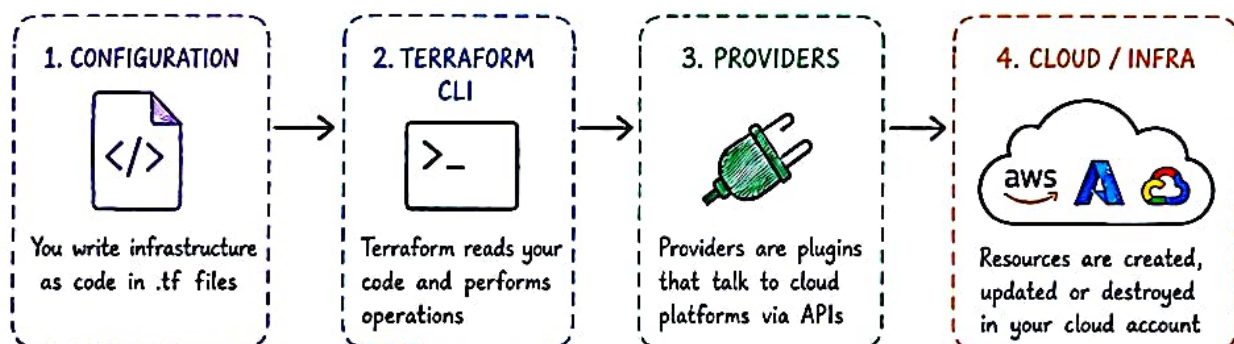
3



Terraform brings
INFRASTRUCTURE
UNDER VERSION CONTROL
so you can build, change
and scale with confidence.

WITHOUT TERRAFORM	WITH TERRAFORM
✗ Manual & error-prone	✓ Automated & consistent
✗ Hard to track changes	✓ Version controlled
✗ Not repeatable	✓ Repeatable deployments
✗ Disaster recovery is hard	✓ Easy recovery & scaling

TERRAFORM ARCHITECTURE



Terraform is platform-agnostic. Use the same workflow across AWS, Azure, GCP and many more.



IMPORTANT FILES



main.tf
Main configuration file where resources are defined.



variables.tf
Declare input variables to make configs dynamic.



terraform.tfvars
Assign values to variables. (Can be multiple tfvars files)



providers.tf
Configure providers like AWS, Azure, GCP, etc.



.terraform.lock.hcl
Locks provider versions for consistent environments.

EXAMPLE - A SIMPLE EC2 INSTANCE

```
provider "aws" {  
  region = "us-east-1"  
}  
  
resource "aws_instance" "web" {  
  ami           = "ami-0c55b159cbfafa1f0"  
  instance_type = "t2.micro"  
  tags = {  
    Name = "MyTerraformEC2"  
  }  
}
```



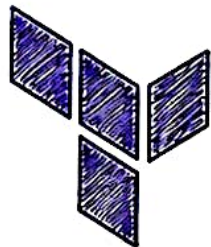
This will launch a t2.micro EC2 instance in us-east-1 region.



KEY TAKEAWAY

Terraform helps you manage infrastructure like code
— consistent, repeatable and reliable.





TERRAFORM

WORKFLOW OVERVIEW

4



Terraform follows a declarative model. You declare the desired state, Terraform does the rest!

1 CREATE / CONFIGURE



Write your infrastructure in .tf files using Terraform Configuration Language (HCL).

2 PLAN



Terraform analyzes your code and shows the execution plan. (What will be created, changed or destroyed)

3 APPLY



Terraform asks for confirmation and provisions the resources.

4 DESTROY



When no longer needed, Terraform destroys the resources and cleans up.



PROVIDERS

Providers are plugins that allow Terraform to interact with cloud platforms or other APIs.



...

AWS | Azure | Google Cloud | and more

Example (AWS Provider)

```
provider "aws" {  
  region = "us-east-1"  
}
```



RESOURCES



Resources are the infrastructure objects you create and manage. (EC2, S3, VPC, RDS, etc.)

Example (AWS EC2 Instance)

```
resource "aws_instance" "web" {  
  ami           = "ami-0c5b159cbfafa1f0"  
  instance_type = "t2.micro"  
  tags = {  
    Name = "MyTerraformEC2"  
  }  
}
```



Resources are the building blocks. You define them, Terraform creates them.

BASIC TERRAFORM WORKFLOW (DEEP DIVE)



A. INIT

Initializes your working directory. Downloads providers, modules, and sets up the backend.

terraform init



B. VALIDATE

Validates the syntax of your configuration files.

terraform validate



C. PLAN

Creates an execution plan and shows what Terraform will do.

terraform plan



D. APPLY

Applies the changes required to reach the desired state.

terraform apply



E. DESTROY

Destroys all resources managed by this configuration.

terraform destroy



- Terraform works on the concept of Desired State vs Actual State.
- It will only make changes when required.
- Your infrastructure is version controlled just like application code.





WRITING YOUR FIRST TERRAFORM CONFIGURATION

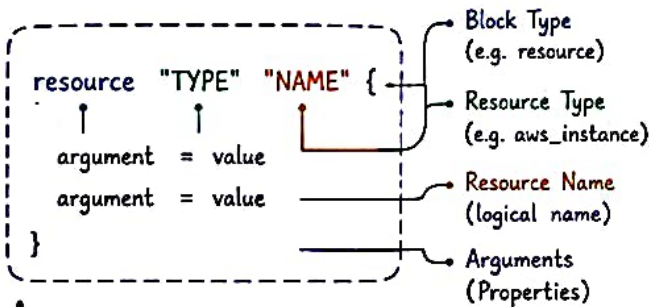


Terraform uses HCL (HashiCorp Configuration Language) to define infrastructure in a simple, readable way.

Everything in Terraform is defined in **BLOCKS**.



1 ANATOMY OF A BLOCK



Key Points

- ✓ Blocks describe what infrastructure you want.
- ✓ Arguments set the desired state for that infrastructure.
- ✓ Terraform figures out how to make it happen.

2 EXAMPLE: EC2 INSTANCE

```
resource "aws_instance" "web" {
  ami           = "ami-0c55b159cbf1e1f0"
  instance_type = "t2.micro"
  tags = {
    Name = "MyTerraformEC2"
    Environment = "Dev"
  }
}
```



3 COMMON BLOCK TYPES



resource

Defines infrastructure objects.

e.g. aws_instance, aws_s3_bucket



data

Fetches information from existing resources.

e.g. aws_ami, aws_caller_identity

(X) variable

Input variables make configs reusable.

e.g. instance_type, region



output

Shows information about resources after apply.

e.g. public_ip, endpoint



provider

Defines the cloud provider and credentials.

e.g. aws, azure, google

4 MINIMAL WORKING EXAMPLE



my-terraform/

main.tf

variables.tf

outputs.tf

terraform.tfvars

- main.tf
Main configuration (resources, providers, etc.)
- variables.tf
Declare input variables
- outputs.tf
Display output values after apply
- terraform.tfvars
Provide values for variables

5 BEST PRACTICES



Keep configurations modular
Break into modules for reuse.



Use variables
Avoid hardcoding values.



Never commit secrets
Use variables and environment variables.



Tag your resources
Helps in organization and cost management.

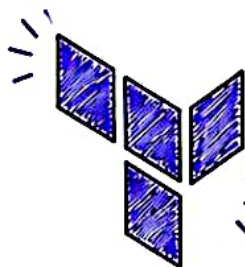


Validate and plan always
terraform validate, terraform plan before apply.



You describe your infrastructure in code,
Terraform makes it real!





TERRAFORM PROVIDERS



Providers are the bridge between Terraform and Cloud Platforms (APIs).

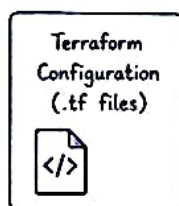


Providers are plugins that allow Terraform to communicate with cloud platforms and services like AWS, Azure, GCP, and many more.

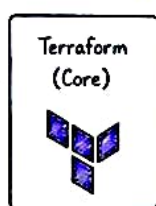
aws



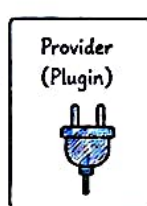
1 HOW PROVIDERS WORK



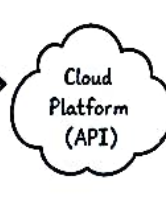
1. You write the configuration.



2. Terraform core reads it.



3. Provider plugin handles the API communication.



4. Cloud platform creates/updates the resources.

POPULAR PROVIDERS



AWS Provider
hashicorp/aws



Azure Provider
hashicorp/azurerm



Google Cloud Provider
hashicorp/google



Kubernetes Provider
hashicorp/kubernetes

... and many more !

2 DECLARING A PROVIDER

Every Terraform configuration starts with a provider block.

```
provider "aws" {
  region = "us-east-1"
  profile = "default"
}
```

★ You can define multiple providers in the same configuration.

3 PROVIDER CONFIGURATION OPTIONS

OPTION	DESCRIPTION
region	Specifies the region where resources will be created.
profile	Uses a named profile from your AWS credentials file (~/.aws/credentials).
access_key / secret_key	Access keys for authentication (Not recommended for production).
assume_role	Allows Terraform to assume an IAM role.
default_tags	Apply common tags to all resources created by the provider.

4 EXAMPLE - MULTIPLE PROVIDERS

```
provider "aws" {
  region = "us-east-1"
}

provider "azurerm" {
  features {}
}

provider "google" {
  project = "my-gcp-project"
  region = "us-central1"
}
```



Terraform can manage resources across multiple clouds in the same configuration!

5 BEST PRACTICES



Use version constraints
Lock provider versions for consistency.



Store credentials securely
Use IAM roles or secret managers.
Avoid hardcoding.



Use default_tags
Keep tagging consistent across all resources.



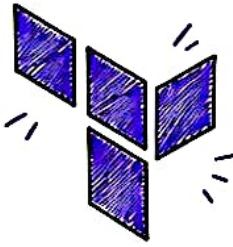
Use providers efficiently
Don't create multiple provider blocks for the same purpose.



Keep provider docs handy
Always refer to the official documentation for updates.



Providers are the power behind Terraform. Learn them deeply, and you can automate anything!



TERRAFORM RESOURCES

7

Resources are the building blocks of your infrastructure. Terraform creates and manages them for you. ★



A resource represents a real infrastructure object like an EC2 instance, S3 bucket, VPC, database, etc. You define it in your configuration and Terraform makes it real in the cloud.

1 ANATOMY OF A RESOURCE BLOCK

```
resource "PROVIDER_RESOURCE_TYPE" "NAME"
  argument1 = value1
  argument2 = value2
  ...
  argumentN = valueN
}
```

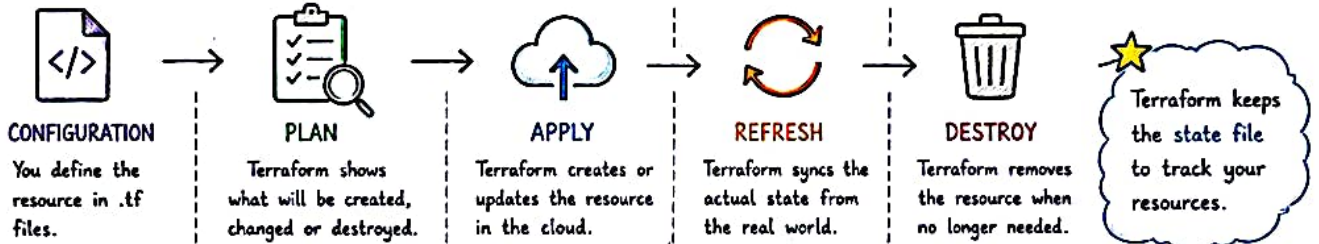
Resource type (e.g. aws_instance)
Resource name (Logical name)
Arguments / Properties

2 EXAMPLE - EC2 INSTANCE

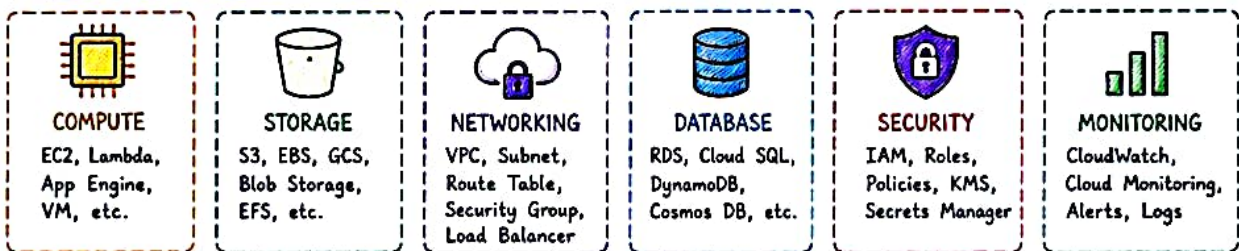
```
resource "aws_instance" "web" {
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = "t2.micro"
  tags = {
    Name        = "TerraformEC2"
    Environment = "Dev"
  }
}
```



3 RESOURCE LIFECYCLE (FLOW)



4 COMMON RESOURCE TYPES



5 META-ARGUMENTS YOU'LL USE OFTEN

#	count	Create multiple instances of a resource. count = 2
☁️	for_each	Create multiple instances using a map or set. for_each = { a = "1", b = "2" }
🔗	depends_on	Define explicit dependencies between resources. depends_on = [aws_iam_role.example]
🔄	lifecycle	Customize resource behavior. lifecycle { create_before_destroy = true }
☁️	provider	Use a specific provider configuration. provider = aws.us_east

6 BEST PRACTICES

- ✓ Give meaningful names to resources.
- ✓ Use variables for values.
- ✓ Keep your configurations modular.
- ✓ Tag your resources for organization.
- ✓ Follow least privilege for security.
- ✓ Review the plan before you apply.



Resources are the heart of your infrastructure. Master them, and you master Terraform! ❤️





TERRAFORM VARIABLES

8



Variables make
Terraform
FLEXIBLE and
REUSABLE.



Variables allow you to parameterize your configurations so you can use the same code for different environments.

1 WHY USE VARIABLES ?

- ✓ Avoid hardcoding values
- ✓ Make configurations flexible
- ✓ Reuse the same code in multiple environments
- ✓ Easily override values
- ✓ Keep secrets and configs separate from code



2 DECLARING A VARIABLE

```
variable "instance_type" {  
  description = "Type of EC2 instance"  
  type        = string  
  default     = "t2.micro"  
}
```

Default
value



Type helps validate input and catches errors early.

3 USING A VARIABLE

```
resource "aws_instance" "web" {  
  ami           = var.ami_id  
  instance_type = var.instance_type  
  tags = {  
    Name = var.name  
  }  
}
```



Variables
make your
config dynamic!

Access variables using:
var.<variable_name>

4 WAYS TO ASSIGN VALUES



1. default (in variable block)
Used when no other value is provided.



2. CLI (-var)
terraform apply -var="instance_type=t3.small"



3. tfvars file
terraform apply -var-file="dev.tfvars"



4. Environment Variables
export TF_VAR_instance_type=t3.small

5 EXAMPLE: TFVARS FILE

dev.tfvars

```
instance_type = "t3.small"  
ami_id        = "ami-0c55b159cbfafef0"  
name          = "web-server"  
region        = "us-east-1"
```

Load it with:

```
terraform apply -var-file="dev.tfvars"
```

6 VARIABLE TYPES

string



"hello"

number

123

42

bool



true / false

list



["a", "b"]

map



{ env = "dev" }

object



{ name = "web" }

7 BEST PRACTICES



Keep sensitive
values in tfvars
or environment
variables.



Use meaningful
names and
descriptions.



Keep variables
modular and
reusable.



Validate inputs
using type and
validation rules.



Don't hardcode
values in your
config.



Variables turn your Terraform code into a powerful, reusable and environment-agnostic infrastructure engine!





TERRAFORM MODULES

9



Modules let you **PACKAGE** and **REUSE** your Terraform configurations. ★



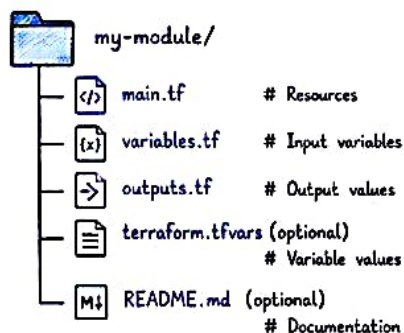
A module is a container for multiple resources that are used together. It helps you write clean, DRY and reusable Terraform code.

1 WHY USE MODULES?

- ✓ Reusability - Use the same code again.
- ✓ Maintainability - Update in one place.
- ✓ Consistency - Standardize your infrastructure.
- ✓ Abstraction - Hide complexity and focus on what matters.



2 MODULE STRUCTURE



This is the standard structure of a custom module.

3 CREATING A SIMPLE MODULE

modules/ec2/main.tf

```
resource "aws_instance" "this" {
  ami           = var.ami
  instance_type = var.instance_type
  tags = {
    Name = var.name
  }
}
```



This module creates an EC2 instance using input variables.

4 USING A MODULE

```
module "web_server" {
  source      = "../modules/ec2"
  ami        = "ami-0c55b159cbf1f0"
  instance_type = "t2.micro"
  name       = "web-server"
}
```



Call the module using the source path and provide values for its variables.

5 INPUT VARIABLES (variables.tf)

```
variable "ami" {
  description = "AMI ID"
  type       = string
}
variable "instance_type" {
  description = "Instance type"
  type       = string
  default    = "t2.micro"
}
variable "name" {
  description = "Name tag"
  type       = string
}
```



Define the inputs your module accepts. Add type, description and default when needed.

6 OUTPUT VALUES (outputs.tf)

```
output "instance_id" {
  description = "ID of the EC2 instance"
  value       = aws_instance.this.id
}
output "public_ip" {
  description = "Public IP of the instance"
  value       = aws_instance.this.public_ip
}
```



Outputs allow you to return useful values from the module.

7 MODULE SOURCES

 Local Path	source = "../modules/ec2"
 Git Repository	source = "git::https://github.com/user/repo.git//module/path"
 Terraform Registry	source = "registry.terraform.io/namespace/module/provider"
 Remote (HTTP)	source = "https://example.com/tf-modules/vpc"

8 BEST PRACTICES

- ✓ Keep modules small and focused.
- ✓ Use meaningful names.
- ✓ Document inputs and outputs.
- ✓ Follow a consistent structure.
- ✓ Version your modules.
- ✓ Test your modules.
- ✓ Avoid hardcoding values.
- ✓ Use variables and outputs to stay flexible.



Modules are the secret to scalable and maintainable Terraform code. Build once. Reuse everywhere. Automate like a pro!

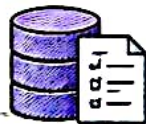




TERRAFORM STATE



The state file is the brain of Terraform. It maps your real infrastructure to your configuration. ★



Terraform state is a JSON file that keeps track of your resources. It helps Terraform know what exists, what changed, and what to update.

1 WHY STATE IS IMPORTANT

- ✓ Tracks resources Terraform created.
- ✓ Maps configuration to real-world resources.
- ✓ Improves performance (no need to query cloud).
- ✓ Enables planning and change detection.
- ✓ Essential for updates and destroys.



2 WHERE IS STATE STORED?

Default (Local)



terraform.tfstate
stored in your
working directory.

Remote (Recommended)

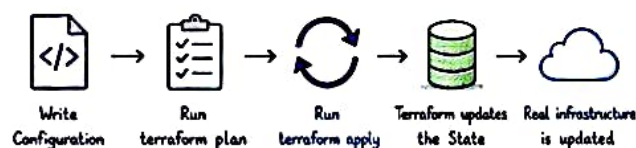


Stored in remote backends
like S3, Azure Blob,
GCS with locking.



Remote state enables collaboration and prevents conflicts.

3 STATE WORKFLOW



State is updated after a successful apply.

4 STATE FILE CONTENT (EXAMPLE)

```

{
  "version": 4,
  "terraform_version": "1.7.0",
  "resources": [
    {
      "type": "aws_instance",
      "name": "web",
      "instances": [
        {
          "attributes": {
            "id": "i-0c55b159cbfafelf0",
            "instance_type": "t2.micro",
            "public_ip": "3.23.45.67"
          }
        }
      ]
    }
  ]
}
  
```



Never edit the state file manually!

Let Terraform handle it.

5 COMMON STATE COMMANDS

COMMAND	DESCRIPTION
terraform state list	List all resources in the state.
terraform state show <resource>	Show details of a specific resource.
terraform state mv <src> <dest>	Move/rename a resource in the state.
terraform state rm <resource>	Remove a resource from the state.
terraform refresh	Sync state with real infrastructure.

6 BACKEND EXAMPLE (S3)

```

terraform {
  backend "s3" {
    bucket = "my-terraform-state"
    key    = "prod/terraform.tfstate"
    region = "us-east-1"
    dynamodb_table = "terraform-locks"
    encrypt      = true
  }
}
  
```

S3 bucket for state

Lock table (prevents concurrency)

7 BEST PRACTICES



Use remote state with locking (DynamoDB, etc.).



Enable encryption for state at rest and in transit.



Control access with IAM policies.



Backup your state regularly.



Never edit state files manually.



Keep state files out of version control.

8 TROUBLESHOOTING TIPS



State out of sync? → Run 'terraform refresh'.



Resource missing in state? → Use 'terraform import'.



Accidental deletion? → Check backups if available.



Lock issues? → Check DynamoDB lock table.



State connects your code to real infrastructure. Protect it, back it up, and let Terraform do the magic! ✨





TERRAFORM PROVISIONING

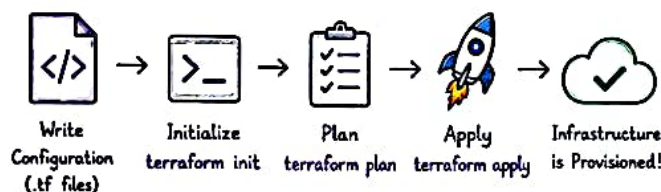


Provisioning is the process of creating infrastructure resources in the right order and making them ready to use. ★



Terraform provisioning creates, manages, and destroys cloud and on-prem resources in a predictable and repeatable way.

① PROVISIONING WORKFLOW



Plan shows what will be created/changed. Apply makes it real!

② PROVISIONING PROCESS



1. Read configuration files
Terraform reads your .tf files.



2. Build dependency graph
Terraform understands resource dependencies.



3. Plan execution
It calculates the execution plan.

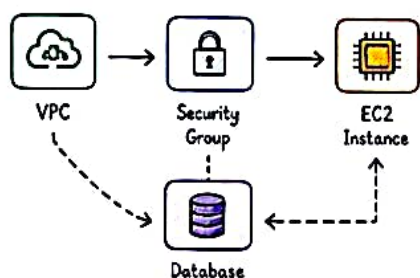


4. Create resources
Resources are created in the correct order.



5. Verify & store state
State file is updated with real infrastructure info.

③ RESOURCE DEPENDENCIES



Terraform creates resources in the right order based on their dependencies.

④ PROVISIONING MODES



Declarative (Terraform Way)

- Define the desired state
- Terraform figures out how to get there
- Idempotent – same result every time



Imperative (Manual Way)

- Step-by-step commands
- You manage the order
- Harder to maintain and repeat



⑤ EXAMPLE: PROVISIONING AN EC2 INSTANCE (AWS)

```

main.tf (Simplified)
resource "aws_instance" "web" {
  ami           = "ami-0c55b199cbfafe1f0"
  instance_type = "t2.micro"
  tags = {
    Name = "web-server"
  }
}
  
```



terraform plan
Plan shows EC2 will be created.



terraform apply
Apply creates the EC2 instance.



EC2 Instance is created in AWS.



State file is updated.

⑥ PROVISIONING BEST PRACTICES

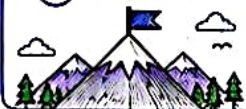
- ✓ Keep configurations modular and reusable.
- ✓ Use variables for flexibility.
- ✓ Use meaningful names and tags.
- ✓ Review the plan before applying.
- ✓ Never edit the state file manually.
- ✓ Store state securely (remote backend).



⑦ COMMON PROVISIONING COMMANDS

COMMAND	PURPOSE
terraform init	Initialize working directory and providers
terraform plan	Show execution plan
terraform apply	Create or update resources
terraform destroy	Destroy all managed resources
terraform taint <res>	Mark resource for recreation
terraform refresh	Sync state with real infrastructure

⑧ KEY TAKEAWAY



- ★ Provisioning is about creating and preparing infrastructure automatically.
- ★ Terraform makes provisioning safe, repeatable, and efficient.
- ★ Write once, provision anywhere!





TERRAFORM DATA SOURCES

12



Data sources allow you to fetch and use information that is already available outside of Terraform management.



Data sources let Terraform read information from existing resources or external APIs.



1 WHAT IS A DATA SOURCE?

- ✓ Reads information from existing resources or external systems.
- ✓ Does not create, update or delete anything.
- ✓ Useful for referencing IDs, ARNs, names, IP addresses, etc.
- ✓ Helps avoid hardcoding values.



2 BASIC SYNTAX

```
data "<PROVIDER>_<TYPE>" "<NAME>" {  
  # Arguments / Filters  
}
```

←--- Read-only block



The data block fetches information using filters, and the result is available in `data.<TYPE>.<NAME>`

3 EXAMPLE: GET EXISTING AWS VPC

```
data "aws_vpc" "main" {  
  filter {  
    name   = "tag:Name"  
    values = ["my-vpc"]  
  }  
}  
  
# Use the data  
output = data.aws_vpc.main.id
```

Fetches the VPC with Name = "my-vpc" and uses its ID.

4 COMMON DATA SOURCES (AWS)

DATA SOURCE	PURPOSE
aws_vpc	Fetch details of an existing VPC
aws_subnet	Fetch details of a subnet
aws_security_group	Fetch Security Group details
aws_iam_role	Fetch IAM Role details
aws_ami	Fetch AMI information
aws_route53_zone	Fetch Route53 hosted zone

5 ACCESSING DATA SOURCE ATTRIBUTES

```
data "aws_subnet" "public" {  
  id = "subnet-12345678"  
}  
  
resource "aws_instance" "web" {  
  ami           = data.aws_ami.ubuntu.id  
  instance_type = "t2.micro"  
  subnet_id    = data.aws_subnet.public.id  
}
```

Use dot notation to access attributes of data sources.

6 USES OF DATA SOURCES

- 🎯 Reference existing infrastructure.
- 📄 Avoid hardcoding IDs and ARNs.
- 🔄 Use outputs from other accounts or modules.
- 🔧 Integrate with resources managed outside Terraform.
- 📊 Fetch dynamic information at plan/apply time.

7 EXAMPLE: GET LATEST AMI

```
data "aws_ami" "ubuntu" {  
  most_recent = true  
  owners     = ["099720109477"] # Canonical  
  
  filter {  
    name   = "name"  
    values = ["ubuntu/images/hvm-ssd/ubuntu-focal-20.04-amd64-server-*"]  
  }  
}  
  
# Use it  
output = data.aws_ami.ubuntu.id
```

Fetches the most recent Ubuntu 20.04 AMI owned by Canonical.

8 BEST PRACTICES

- ✓ Use meaningful names for data blocks.
- ✓ Filter precisely to get the correct resource.
- ✓ Prefer tags for filtering.
- ✓ Handle multiple results carefully.
- ✓ Document why a data source is used.
- ✓ Combine with variables for flexibility.



Data sources give your Terraform code superpowers:
Use what already exists and build even better! ✨





TERRAFORM OUTPUTS & LOCALS

13



Outputs expose important information from your Terraform code. Locals help you reuse values and keep your code DRY.



Outputs = Share important info outside Terraform.
Locals = Store and reuse values inside Terraform.

1 WHY USE OUTPUTS & LOCALS?

OUTPUTS

- ✓ Show important info after apply
- ✓ Share values with other Terraform configurations
- ✓ Useful for scripts, users, or other teams



LOCALS

- ✓ Avoid repetition
- ✓ Make complex values easier to manage
- ✓ Improve readability and maintainability



2 OUTPUTS IN TERRAFORM

```
output "instance_public_ip" {  
  description = "Public IP of the EC2 instance"  
  value       = aws_instance.web.public_ip  
}  
  
output "instance_id" {  
  value = aws_instance.web.id  
}
```

This will be shown after 'terraform apply'



Outputs are read-only. They do not change infrastructure.

3 HOW TO USE OUTPUTS?



terraform output
Show all outputs



terraform output <name>
Show a specific output



terraform output -json
Show outputs in JSON format



Use in other config
Via remote state or scripts

Example Output

```
$ terraform output  
instance_id = "i-0c55b159cbfafa1f0"  
instance_public_ip = "3.23.45.67"
```



Outputs are saved in the state file.

4 LOCALS IN TERRAFORM

```
locals {  
  common_tags = {  
    Environment = "dev"  
    Project      = "website"  
    Owner        = "devops-team"  
  }  
  
  instance_name = "web-server-01"  
}
```



Locals are like variables, but auto-computed inside your configuration.

5 USE LOCALS IN RESOURCES

```
resource "aws_instance" "web" {  
  ami           = "ami-0c55b159cbfafa1f0"  
  instance_type = "t2.micro"  
  tags = merge(local.common_tags, {  
    Name = local.instance_name  
  })  
}
```



Using locals keeps your configuration clean and consistent.

6 OUTPUTS + LOCALS TOGETHER (EXAMPLE)

```
locals {  
  env           = "dev"  
  common_tags   = Environment = local.env, Project = "shop"  
}  
  
resource "aws_instance" "app" {  
  ami           = "ami-0c55b159cbfafa1f0"  
  instance_type = "t2.micro"  
  tags          = merge(local.common_tags, { Name = "app-server" })  
}  
  
output "app_public_ip" {  
  value       = aws_instance.app.public_ip  
  description = "Public IP of the App Server"  
}
```



Clean code

Clear code
• Useful
• Information



7 BEST PRACTICES



Output only what is necessary.



Do not output sensitive values (passwords, keys).



Use locals to store repeated or complex values.



Keep names meaningful and descriptive.



Document your outputs with descriptions.



Use outputs for integration with other systems.

8 COMMON USE CASES



Share outputs with other Terraform configurations (Remote State)



Pass values to scripts or automation tools



Give important info to DevOps or operations teams



Display info for users after deployment

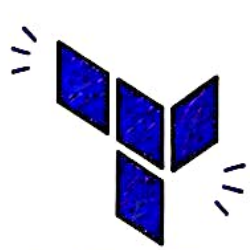


Use in CI/CD pipelines for verification and reporting



Outputs make your infrastructure useful to the real world.
Locals make your code clean, smart and maintainable.





TERRAFORM

EXPRESSIONS & FUNCTIONS

14



Expressions let you combine values, variables and functions to build dynamic, flexible and powerful Terraform configurations.



Expressions = Combine values, variables, literals and functions.
Functions = Built-in helpers to transform and manipulate values.

1 WHAT IS AN EXPRESSION?

- ✓ An expression produces a value.
- ✓ You can use literals, variables, references or functions.
- ✓ Terraform evaluates expressions during the plan / apply.



Example:

```
var.instance_type == "t2.micro" ? 1 : 0  
(Ternary Expression)
```

2 BASIC EXPRESSIONS

EXPRESSION	EXAMPLE
Literals	"hello", 123, true
References	var.name, local.env
Attribute Access	aws_instance.web.id
Index / Splat	aws_subnet.public[0].id aws_instance.web[*].id
Ternary (Condition)	var.env == "prod" ? "yes" : "no"
Template (String)	"Hello, \${var.name}!"
For Expressions	[for s in var.subnets : s.id]

3 TYPES OF FUNCTIONS



Terraform has 100+ built-in functions.
Use them to simplify and make your code smarter.

4 STRING FUNCTIONS (EXAMPLES)

FUNCTION	DESCRIPTION	EXAMPLE	RESULT
upper(str)	Convert to upper case	upper("devops")	DEVOPS
lower(str)	Convert to lower case	lower("DEVOPS")	devops
title(str)	Title case	title("hello world")	Hello World
trim(str)	Remove spaces	trim(" hi ")	hi
substr(str, s, l)	Substring	substr("abcdef", 1, 3)	bcd
join(sep, list)	Join list to string	join(", ", ["a", "b"])	a,b
format(str, ...)	Formatted string	format("Hi %s", "Tom")	Hi Tom

5 NUMERIC FUNCTIONS (EXAMPLES)

FUNCTION	DESCRIPTION	EXAMPLE	RESULT
abs(n)	Absolute value	abs(-10)	10
max(a, b, ...)	Maximum value	max(2, 5, 3)	5
min(a, b, ...)	Minimum value	min(2, 5, 3)	2
ceil(n)	Round up	ceil(2.3)	3
floor(n)	Round down	floor(2.8)	2
pow(a, b)	a to the power b	pow(2, 3)	8

6 COLLECTION FUNCTIONS (EXAMPLES)

FUNCTION	DESCRIPTION	EXAMPLE	RESULT
length(list/map)	Length of list or map	length([1,2,3])	3
contains(list, v)	Check if value exists	contains(["a","b"], "a")	true
tolist(set)	Convert set to list	tolist(["a","b"])	["a","b"]
tomap(object)	Convert object to map	tomap({a=1,b=2})	{a=1,b=2}
merge(m1, m2, ...)	Merge maps	merge({a=1},{b=2})	{a=1,b=2}
keys(map)	All map keys	keys({a=1,b=2})	["a","b"]
values(map)	All map values	values({a=1,b=2})	[1,2]

7 FOR EXPRESSIONS

Build new lists or maps from existing collections.

Example - List

```
[for s in var.subnets :  
  s.id]
```

Returns list of subnet IDs

Example - Map

```
{ for k, v in var.tags :  
  k => upper(v) }
```

Returns a map with upper case values



Great for dynamic and repeatable configurations.

9 NESTED EXPRESSIONS (EXAMPLE)

```
resource "aws_s3_bucket" "logs" {  
  bucket = format("%s-logs-%s", local.project_name, var.env)  
  tags = merge(local.common_tags, {  
    Environment = var.env  
  })  
}
```



Combine functions, locals and variables for powerful results.

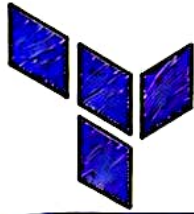
10 BEST PRACTICES

- ✓ Keep expressions simple and readable.
- ✓ Use locals for complex or repeated values.
- ✓ Prefer functions over hardcoding.
- ✓ Add meaningful names and comments.
- ✓ Test with 'terraform console'.
- ✓ Validate with 'terraform plan'.
- ✓ Use outputs to verify values.



Expressions add intelligence to your Terraform code.
Functions save time, reduce errors and improve reusability.





TERRAFORM

DEPENDENCIES & LIFECYCLE

15



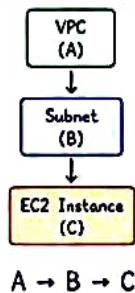
Control the order of resource creation and manage how Terraform creates, updates and destroys your infrastructure with Lifecycle rules.



Terraform builds a dependency graph automatically, but you can also define explicit dependencies. Lifecycle rules give you fine-grained control over resource behavior.

1 HOW DEPENDENCIES WORK

- ✓ Terraform automatically detects dependencies by references.
- ✓ If Resource B uses an attribute of Resource A, A will be created before B.
- ✓ Sometimes you need to define explicit dependency.



2 EXPLICIT DEPENDENCY

```
resource "null_resource" "example" {
  depends_on = [aws_iam_role.app_role]

  provisioner "local-exec" {
    command = "echo IAM role created!"
  }
}
```

depends_on ensures this resource is created only after aws_iam_role.app_role is created.



Use depends_on only when Terraform cannot automatically detect the dependency.

3 DEPENDENCY EXAMPLES

SCENARIO	HOW TERRAFORM HANDLES IT
Subnet uses VPC ID	Automatically creates VPC before Subnet.
EC2 uses Security Group	Creates Security Group before EC2.
S3 Bucket used in Lambda environment	Creates S3 Bucket before Lambda.
Resources with no direct reference	Use depends_on to set explicit order.

4 LIFECYCLE RULES OVERVIEW

RULE	PURPOSE
create_before_destroy	Create new resource before destroying the old one.
prevent_destroy	Prevent accidental destruction of the resource.
ignore_changes	Ignore changes to specific attributes.
replace_triggered_by	Trigger replacement when another resource or attribute changes.



Protect



Control



Customize

5 CREATE BEFORE DESTROY EXAMPLE

```
resource "aws_db_instance" "db" {
  identifier = "mydb"
  engine    = "mysql"
  instance_class = "db.t3.micro"

  lifecycle {
    create_before_destroy = true
  }
}
```



Terraform will create the new DB instance first, then destroy the old one.
Reduces downtime!

6 PREVENT DESTROY EXAMPLE

```
resource "aws_s3_bucket" "important" {
  bucket = "my-important-data"

  lifecycle {
    prevent_destroy = true
  }
}
```



Terraform will block any action that would destroy this bucket.
Very useful for production resources!

7 IGNORE CHANGES EXAMPLE

```
resource "aws_instance" "app" {
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = "t2.micro"
  tags = {
    Name = "app-server"
    Env  = "dev"
  }

  lifecycle {
    ignore_changes = [tags["Name"], instance_type]
  }
}
```



Changes to these attributes will be ignored and will not trigger updates.

8 REPLACE TRIGGERED BY EXAMPLE

```
resource "aws_launch_template" "lt" {
  name     = "app-lt"
  image_id = "ami-0c55b159cbfafe1f0"
  instance_type = "t3.micro"
}

resource "aws_autoscaling_group" "asg" {
  name     = "app-asg"
  launch_template {
    id      = aws_launch_template.lt.id
    version = "Latest"
  }

  lifecycle {
    replace_triggered_by = [aws_launch_template.lt]
  }
}
```



If the Launch Template changes, Terraform will replace the ASG.

9 COMMON PITFALLS

- ✗ Overusing depends_on can make configs harder to maintain.
- ✗ prevent_destroy can block important changes if forgotten.
- ✗ Ignoring changes may hide real drifts.
- ✗ create_before_destroy may cause temporary cost increase.
- ✗ Learn how Terraform builds the graph - it's smart!



10 BEST PRACTICES

- ✓ Let Terraform infer dependencies automatically when possible.
- ✓ Use depends_on only when absolutely necessary.
- ✓ Use lifecycle rules to protect and control critical resources.
- ✓ Test changes in a non-production environment.
- ✓ Document why a lifecycle rule is used.



Dependencies make sure resources are created in the correct order.
Lifecycle rules give you control, safety and reliability in your infrastructure!

